JavaScript-01 第9回_まとめ

第9回の目標

- **JavaScriptの基礎を知る**
- **変数・条件分岐・関数の書き方に慣れる**
- **DOM操作でHTMLをJSから動かす感覚をつかむ**

JavaScript

主にブラウザ上で動作するプログラミング言語。 Web制作におけるJavaScriptの役割は、基本的には「Webサイトに動きをつける」こと（ふわっとしたアニメーション、ハンバーガーメニューの開閉など）。 - ユーザーが入力した数値に応じて計算する - クリックした項目によって表示を切り替える - スクロールやクリック、リロードをきっかけにアクションを起こす

⇒ **結論：CSSだけでは実現できない、ユーザーの操作に応じた「動的」な処理ができる。**

他にもいろいろなところに使われる - Webアプリケーション - スマホアプリケーション - サーバーサイド

📎 JavaScriptで何が出来るか学ぼう

JavaScriptを書く準備

準備の流れはCSSと同じ。相対パス・ディレクトリ（＝フォルダのこと）は大事な概念なので、自信がなければ調べておく。

①ファイルを用意する

拡張子は`.js`。多くの現場では`js`フォルダの中にjsファイルをまとめる構成が一般的（例：`project/js/main.js`）。

②読み込み場所を記述する

HTMLファイルに「相対パス」で「scriptタグ」を使って読み込む（headタグ内に書くことが多い）。

```
<script src="js/main.js"></script>
```

- `script`タグは`head`の終了タグの直前に書く

- よくある間違いは相対パスの理解不足（📎 相対パスと絶対パスの違い）

③defer属性をつける

DOM（HTMLの要素）を読み込み終えてからJSを実行するよう指示する属性。表示速度の面でもメリットがあるため、基本的につけておく。

```
<script src="js/main.js" defer></script>
```

📎 defer属性についての参考サイト／deferとasyncの違い（図解）

④コンソール画面を開く

エラーを表示してくれる便利なツール。行の右端に、エラーが出ているファイルと行数も表示される。 - 右端にJSのファイル名・行数が出ていないエラー（例：`WebSocket connection ... failed`）は、JSとは関係ないことが多いので気にしなくてよい

OS	ショートカットキー
Windows	Ctrl + Shift + J
Mac	⌘ + Option + J

検証ツール → Consoleタブからでも同様に確認できる。

エラーをAIに聞くとき

エラー文の意味が分からないときは、AIに次のように聞くとよい。

```
下記のエラー文の解説と対策を教えてください。
プログラミング初心者にもわかるように具体例も交えてください。

##ここにエラー文
```

- AIにコードを書かせ続けると、本当に困ったときに自分で直せなくなる。あくまで理解の補助に使う

データ型

- プリミティブ型（単一の値を使う）
- 文字列（string）
  - " "で囲まれたテキスト

- 数値（Number）
  - 計算を行う数値を扱う
- 真偽値（Boolean）
  - 「true」と「false」という 2 つの値が入る
- undefined
  - 変数の中身が未定義の状態だと「undefined」の値を返す
- null
  - 意図的にオブジェクトの値が存在していないことを表す
- オブジェクト型（複数の値を使う）
- オブジェクト（Object）
- 配列（Array）
- 関数（Function）
- 日付（Date）
- 正規表現（RegExp）
- JSONオブジェクト

変数

変数の宣言と代入

変数を定義する際には必ず「変数の宣言」を行う。 「宣言のキーワード（`let`／`const`） 変数名 = 値;」の形で変数を宣言し、値を代入する（授業ではキーワードの部分を「変数の型」と呼んでいる）。 - 変数は、数値や値などを入れておく箱のようなもの。何度も使う値をまとめておける（CSS変数と同じ考え方） - `=`は「等しい」ではなく「右の値を左の変数に**代入**する」という意味 - 「等しい」を表すときは`==`か`===`を使う - `console.log()`：かっこ内の値をコンソール画面に表示する。動作確認に使う - VSCodeでは「log」と打って↓を1回押し、Enterで出せる

```
let score = 80;
console.log(score);
```

再代入

一度宣言した変数に別の値を入れ直すことを「再代入」と呼ぶ。「再定義（もう一度`let`などをつけて宣言し直すこと）」とは別物なので混同しないように注意する。

```
let score = 80;
console.log(score);


```

```
// 再代入
score = 95;
console.log(score);
```

書き方	呼び方
`let fruit = "りんご";`	宣言（定義）
`fruit = "ぶどう";`	再代入
`let fruit = "りんご";` の後に `let fruit = "ぶどう";`	再宣言 → **エラー**（同じ名前を2回宣言している）

文字列と変数の書き分け

- 文字列は`"`や`'`で囲う。変数名は囲わない

```
let fruit = "りんご";

console.log(fruit);   // 「りんご」（変数の中身）
console.log("fruit"); // 「fruit」（ただの文字列）
```

- `"`と`'`はどちらでもよい。JSではシングルクォートが使われることが多い（HTMLのダブルクォートと区別するため。ESLintや一般的なスタイルガイドでも推奨されることが多い）。大事なのは**統一すること**
  - 文字列の中で`'`を使うときは、外側を`"`にする（例：`"That's a good idea"`）
- `;`（セミコロン）は処理の終わりを表す（文章の「。」）。VSCodeの拡張機能 Prettierで自動で入れられる

数値と文字列の違い

```
const a = 50;   // 数値
const b = "50"; // 文字列

console.log(a + 2); // 52（足し算）
console.log(b + 2); // 502（文字として連結される）
```

- コンソールでは、数値は紫色、文字列は白色で表示される

テンプレートリテラル

文字列の中に変数を埋め込む書き方。`` ` ``（バッククォート）で囲い、変数を`${}`に入れる。

```
const age = 20;
console.log(`${age}歳は、成人です`);      // テンプレートリテラル
console.log(age + "歳は、成人です");      // +で連結しても同じ
```

- `'`や`"`ではなく`` ` ``で囲うので注意
- 変数を1つ出すだけなら、テンプレートリテラルは使わなくてよい（`console.log(i);`で十分）
- 「これは何？」と思ったら、「js ドルマーク 括弧」のように言語化して検索する（ テンプレートリテラルとは）

let・const・varの違い

種類	呼び方	再代入	再定義
let	変数	⭕	❌
const	定数	❌	❌
var	（古い書き方）	⭕	⭕

昔は`var`が主流だったが、再定義まで可能なため長いコードや複数人での開発で意図しない上書きが起きやすかった。そこで「再代入はしたいが再定義はしたくない`let`」「再代入すらしたくない`const`」が使われるようになり、現在は`let`・`const`が主流になっている。 - 古いサイトで`var`が使われていたら、`let`や`const`に書き換えるとよい - `const`は1度定義すると中身を変えられない（意外と便利）。これが`let`との大きな違い

constとletどちらを使う？

- 基本は`const`を使い、再代入が必要なものだけ`let`にする
- 最初はどれが再代入が必要か分からないので、慣れるまでは全部`let`でもよい
- 実際は1から書くより、調べて出てきたコードをコピペすることが多い。最初から`const`か`let`で書かれているので、自分で判断する場面はあまりない

変数を使う理由

- 変数を定義しても、ページの見た目には直接影響しない。変数を使わなくてもアニメーションは作れる（CSS変数と同じ）
- JSで要素を操作するときは、毎回`document.querySelector(".クラス名")`を書く必要がある
  - 何回も書くとコードが長くなる
  - クラス名が変わったとき、すべての箇所を書き換える必要がある
- `const box = document.querySelector(".js_box");`のように変数に入れておけば、以降は`box`と書くだけで操作できる

JavaScriptの変数を学ぼう

条件分岐（if文）

基本の書き方

```
if (条件1) {
  // 条件1が真のときの処理
} else if (条件2) {
  // 条件2が真のときの処理
} else {
  // どちらにも当てはまらないときの処理
}
```

条件は上から順番に判定されるため、範囲を絞り込むような条件を並べるときは判定の順序に注意する（最も厳しい条件を先に書くなど）。

書き方	日本語にすると
`if`	もし〇〇だったら、〜〜する
`else if`	そうじゃなくて△△だったら、〜〜する
`else`	それ以外だったら、〜〜する

処理の進み方 1. `()` 内の条件を確認する 2. 当てはまったら、`{}` 内の処理を実行する 3. 当てはまらなかったら、次の `()` 内の条件を確認する（1に戻る）

押さえるポイント：構文（書き方の型）、処理の仕組み、演算子、`else` と `else if` の使い分け

条件が狭いものから書く

```
const score = 100;
if (score >= 80) {
  console.log("合格です！");
} else if (score >= 60) {
  console.log("もう少しで合格です！");
} else if (score >= 30) {
  console.log("もう少し頑張りましょう！");
} else {
  console.log("学習範囲を反復しましょう！");
}
```

- `score >= 30`を先に書くと、90点でも最初の条件に当てはまってしまう

elseで済むところはelseにする

```
if (age < 20) {
  console.log("未成年です");
} else {              // else if (age >= 20) でも間違いではないが、
  console.log("成人です"); // age < 20 でなければ20以上が確定しているのでelseの方がシンプル
}
```

if文のスニペット

VSCodeの「スニペット」→「javascript.json」に登録できる。慣れないうちは使ってもよい。

```
"IfTemplate": {
  "prefix": "if-template",
  "body": "if (条件式$1) {\n  処理$2;\n} else if (条件式$3) {\n  処理$4;\n} else {\n  処理$5;\n}",
  "description": "if文のテンプレート"
}
```

- if文の書き方/使い方／JavaScriptで扱えるデータの種類

演習：年齢で出し分ける

```
const age = 20;


if (age < 0 || isNaN(age)) {
  console.log("正しい値を入力してください。");
} else if (age < 20) {
  console.log("あなたは20歳未満です。");
} else if (age >= 20) {
  console.log("あなたは20歳以上です。");
} else {
  console.log("正しい値を入力してください。");
}
```

- `isNaN()`：値が数値でない（Not a Number）ときに`true`を返す関数。チャレンジ問題で「数値かどうか」の判定に使った
- `||`（または）で、2つの条件のどちらかに当てはまれば実行されるようにしている

switch文

1つの値（式）が、どの候補と一致するかで処理を分けたいときに使う。`if〜else if`を何個も並べるより見やすくなる場合がある。 - `case`ごとに`break;`を書かないと、次の`case`の処理まで続けて実行されてしまう - `case`を続けて書くと「AまたはB」の条件にできる - どれにも当てはまらないときは`default`の処理が実行される

```
switch(式){
 case 比較対象1:
  //式が比較対象1と一致した場合に実行する処理
  break;

 case 比較対象2:
  //式が比較対象2と一致した場合に実行する処理
  break;

 case 比較対象3:
 case 比較対象4:
  //式が比較対象3または比較対象4と一致した場合に実行する処理
  break;

 default:
  //どのcaseにも一致しなかった場合に実行する処理
  break;
}
```

比較演算子

演算子	意味
`<` `<=`	未満／以下
`>` `>=`	より大きい／以上
`===`	等しい（型も含めて厳密に比較）
`!==`	等しくない

論理演算子

演算子	意味
`&&`	かつ（両方の条件を満たす）

演算子	意味
`||`	または（どちらかの条件を満たす）
`!`	～でない（条件を反転させる）

算術演算子（よく使うもの）

演算子	意味
`+` `-` `*` `/`	足し算・引き算・掛け算・割り算
`%`	割り算の余り（`i % 3 === 0` なら3の倍数）
`++`	1増やす（`count++`）

🖇️ いろいろな演算子を学ぼう 🖇️ JavaScriptの条件分岐を学ぼう

関数

関数とは

一連の処理をひとまとめにしたもの。「何かを受け取って、何かを返す」マシーンのようなイメージ。 - 同じ処理を何度も使い回したいとき - 任意のタイミングで処理を実行させたいとき

に使う。

- 全自動洗濯機のイメージ：「洗剤を入れる・洗う・すすぐ・脱水する」という多くの処理を、ボタン1つでまとめて動かせる
- 押さえること
    1. 関数の中には処理が入っている
    2. スイッチを入れる（呼び出す）まで動かない
- メリット：何度も使う処理を毎回書くと読みにくく、修正も大変。関数にまとめればどちらも解決する。引数で動きのバリエーション（右から出る・下から出るなど）も変えられる

定義と呼び出し

```
// 関数の定義
function 関数名(引数) {
  // 一連の処理
}
```

```
// 関数の呼び出し
関数名(渡す値);
```

引数と返り値（戻り値）

引数は関数に渡す「材料」。返り値は処理の結果として関数から返ってくる値で、`return`で指定する。引数と返り値はセットで使われることが多い。

```
function introduce(name) {
  const message = name + "さん、こんにちは";
  return message;
}

console.log(introduce("山田"));
```

- `function`は関数を作るときの決まり文句。関数名・引数名は好きな名前でよい
- 引数：同じ命令の一部を変えて使い回したいときに使う「材料」。変えたい部分が複数あれば、引数を増やす
- 返り値：自動販売機のように、関数の結果を呼び出し元に返す値。関数の外で結果を使いたいときに`return`で書く

```
// panmanという関数を宣言する。foodとtypeという材料（引数）を受け取る
function panman(food, type) {
  // nameという箱を用意して、foodとtypeを足したものを代入する
  let name = food + type;

  // 出来上がったものを関数の外に返す
  return name;
}

// 関数のスイッチを押して動かす。入れたい材料をここで渡す
console.log(panman("あん", "パンまん"));      // あんパンまん
console.log(panman("メロン", "パンなちゃん")); // メロンパンなちゃん
console.log(panman("ジャム", "おじさん"));     // ジャムおじさん
```

返り値は電卓の「=」

- 「3+5」と押しても、「=」を押さないと結果は出ない。関数で「3+5」を作り、`return`で「=」を押すイメージ

- 関数の`{}`の中は別世界で、外からは中の変数を使えない（スコープ）。だから`return`で外に出す
- 関数名や返す変数の名前は、処理の内容が分かるものにする（CSSのクラス名と同じ考え方）
- 📎 関数の定義・呼び出し・戻り値の基本

関数の書き方は3種類

```
// 関数宣言
function greeting(name) {
  console.log(`${name}さん、こんにちは`);
}

// 関数式
const greeting2 = function (name) {
  console.log(`${name}さん、こんにちは`);
};

// アロー関数
const greeting3 = (name) => {
  console.log(`${name}さん、こんにちは`);
};

greeting("佐藤"); // 呼び出し
```

- どれでも同じ動き。今すぐ暗記しなくてよく、記事を見ながら真似して書ければよい
- 書き方のルールは会社やチームによって違う。「忘れたら調べればよい」と考える

演習：2つの数を足す関数

```
function sum(a, b) {
  const result = a + b;
  return result;
}

console.log(sum(4, 8)); // 12
```

- 関数作成の流れは「関数の定義（マシーンを作る）→ 中身の処理を記述（動きを設定）→ 関数の呼び出し（スイッチON）」
- 同じ動きをしていればどんな書き方でもよいが、不必要に長い場合は改善の余地がある
- 📎 引数と返り値のイメージ（パン製造マシーンの例）

JavaScriptの関数を学ぼう

DOM操作

DOMとは

DOM（Document Object Model）とは、HTMLの要素をプログラムから自由に操作するための仕組み。JSでHTMLの要素を取得し、操作することを「DOM操作」と呼ぶ。JavaScriptでHTML・CSSを操作することによってアニメーションを作り出すことができる。 - 簡単にいうとHTMLの情報のこと - `document`には、HTML全体の情報が入っている DOMとは（参考）

要素の取得

`document.getElementsByClassName()`

- DOMは配列で取得される（正確には配列に似た`HTMLCollection`という形。`[0]`のように番号で取り出せる）
- "class-name"のように、class属性名をそのまま指定する

```
document.getElementsByClassName("class-name");


const className = document.getElementsByClassName("class-name");
console.log(className[0]);
console.log(className[1]);
console.log(className[2]);
```

`document.getElementById()`

- DOMはそのまま取得される
- "id-name"のように、id属性名をそのまま指定する

```
document.getElementById("id-name");


const idName = document.getElementById('id-name');
console.log(idName);
```

`document.querySelector()`（1つの要素を取得）

- そのまま取得される

- CSSセレクタと同様にclass属性なら.（ドット）、id属性なら#（シャープ）が必要
- 複数ある場合は、最初の要素が取得される

```
document.querySelector(".class-name");
document.querySelector("#id-name");
document.querySelector("div");
document.querySelector("div p");

const queryClassName = document.querySelector('.class-name');
const queryIdName = document.querySelector('#id-name');
console.log(queryClassName);
console.log(queryIdName);
```

`document.querySelectorAll()`（複数の要素を取得）

- 配列で取得される（正確には配列に似た`NodeList`という形。`forEach`で1つずつ処理できる）
- CSSセレクタと同様にclass属性なら.（ドット）、id属性なら#（シャープ）が必要

```
document.querySelectorAll(".class-name");
document.querySelectorAll("div");
document.querySelectorAll("div p");

const queryAllClassName = document.querySelectorAll('.class-name');
const queryAllIdName = document.querySelectorAll('#id-name');

console.log(queryAllClassName);
console.log(queryAllIdName);
```

```
const box = document.querySelector(".sample_box");
console.log(box);
```

JavaScriptで要素を取得しよう

イベントとクラス操作

`addEventListener`でクリックなどのイベントを検知し、`classList`メソッドでクラスの付け外しを行うのが基本の流れ。

```
const box = document.querySelector(".sample_box");

box.addEventListener("click", function () {
  box.classList.toggle("is-active");
});
```

メソッド	動き
`classList.add()`	クラス属性を追加する
`classList.remove()`	クラス属性を削除する
`classList.toggle()`	クラス属性を付け外しする（ついていれば外す、なければ付ける）

見た目の変化はCSS側（`.is-active`がついたときのスタイル）にあらかじめ定義しておき、`transition`を組み合わせるとアニメーションになる。

DOM操作の型

一言一句書けるようになる必要はない。型をメモしておき、`[ ]`の部分を書き換えて使う。

```
// 取得したい要素を、好きな名前の変数に入れる
const [変数名] = document.querySelector(".[取得したいクラス名]");

// 変数に対してイベントが起きたら
[変数名].addEventListener("[イベント]", function () {
  // クラス名を追加する
  [変数名].classList.add("[追加したいクラス名]");
});
```

- `function () {}`は`() => {}`（アロー関数）でもよい
- 見た目の変化は、ほとんど「クラス名の付け替え」で起こしている
  - CSS：変化後の見た目を用意しておく
  - JS：要素を取得 → 変化のタイミングを指定 → クラス名（`is-active`など）を付ける

よく使うイベント

イベント	タイミング
`click`	クリックしたとき
`submit`	フォームを送信したとき

イベント	タイミング
`focus` / `blur`	要素がフォーカスされたとき／外れたとき
`keydown`	キーが押されたとき
`load`	ページが完全に読み込まれたとき
`resize`	ウィンドウサイズが変わったとき
`scroll`	スクロールされたとき
`mouseover` / `mouseout`	マウスが乗ったとき／離れたとき

- addEventListenerの使い方

CSSでis-activeを書くときの注意

書き方	意味
`.js_box.is-active`（スペースなし）	1つの要素に両方のクラスがついているとき。**JSでクラスを追加する場合はこちら**
`.js_box .is-active`（スペースあり）	`.js_box`の子要素で`.is-active`を持つもの

コードを日本語に読み下す

コードを日本語に直して読むと、書くべきことが分かりやすくなる。バグを見つけたり、人に説明したりするのにも役立つ。

```
const box = document.querySelector(".js_box");
// js_boxの要素を1つ取得して、boxという変数に入れる

box.addEventListener("click", function () {
  // boxがクリックされたら
  box.classList.add("is-active");
  // boxにis-activeを追加する（CSSで見た目が変わる）
});
```

要素の取得方法の使い分け

- Web制作では、基本は`querySelector()`（1つ）と`querySelectorAll()`（複数）を使えばよい
- `getElementById()`などはアプリケーション開発でよく使う
- `getElementsByClassName()`はクラス名の頭に`.`がいらない。配列で返ってくるので`[0]`で取り出す

- DOM操作一覧／図解付きDOM解説／jsの命名ルール／イベントオブジェクト（引数e）について（卒業後）

JavaScriptでCSSを追加する

```
const jsFind = document.querySelector(".js_find");
// jsFindの中から .find_list-item-4 を取得する（documentではなく要素から探す）
const findListItem4 = jsFind.querySelector(".find_list-item-4");
// 文字色を直接変える
findListItem4.style.color = "#f00";
```

演習：クリックで変形させる

```
const box02 = document.querySelector(".js_box02");

box02.addEventListener("click", function () {
  box02.classList.toggle("is-active");
});
```

```
.js_box02 {
  transition: 0.5s; /* 変化前の要素につけておくと、付け外しの両方でなめらかに変化する */
}

.js_box02.is-active {
  transform: translateX(200px) rotate(360deg);
}
```

- JSで操作する要素には`js_`の接頭辞をつける（第4回参照）。状態を表すクラスは`is-active`のように`is-`をつけることが多い
- `transition`を`.is-active`側だけに書くと、クラスを外したときには一瞬で元に戻ってしまう
- CSSで変化の途中（経過時間）をつける（transition）

JavaScriptでクラスを操作しよう JavaScriptのイベントでDOM操作をしよう

【＋α】繰り返し処理

授業内では扱わなかったが、同じ処理を繰り返すための構文としてfor文・while文・forEach文がある。余裕があれば予習・復習課題のチャレンジ問題で触れておくとよい。 - **for文**：繰り返す回数があらかじめ決まっている処

理向き - **while文**：条件を満たす間だけ繰り返したい処理向き - **forEach文**：配列の要素を1つずつ取り出して処理したいときに使う

```
// while文：numが15以下の間、繰り返す
let num = 1;
while (num <= 15) {
  console.log(num);
  num++;
}

// for文：(初期値; 条件; 増やし方) を1行で書く
for (let i = 1; i <= 15; i++) {
  console.log(i);
}
```

演習：FizzBuzz

```
for (let i = 1; i <= 30; i++) {
  if (i % 15 == 0) {
    console.log("FizzBuzz!!");
  } else if (i % 3 == 0) {
    console.log("Fizz");
  } else if (i % 5 == 0) {
    console.log("Buzz");
  } else {
    console.log(i);
  }
}
```

- 割った余り（`%`）が0かどうかで、倍数であるかを判定している
- JSの処理は上から進むため、最も条件が厳しい「15の倍数」を一番最初の条件にする

JavaScriptのループ処理を学ぼう JavaScriptのスコープを理解しよう オブジェクトと配列を学ぼう

スコープ

変数を使える（参照できる）範囲のこと。 - **グローバルスコープ**：関数や`{}`の外で宣言した変数。どこからでも参照できる - **ローカルスコープ**：関数や`{}`（ブロック）の中で宣言した変数。その中でしか参照できない

```
const siteName = "ZeroPlus"; // グローバルスコープ

function showName() {
  const message = "こんにちは"; // ローカルスコープ（この関数の中だけ）
  console.log(message + siteName); // どちらも使える
}

showName();
console.log(message); // エラー：関数の外からは参照できない
```

- `let`・`const`は`{}`（if文やfor文のブロック）の中で宣言すると、そのブロックの外では使えない
- JavaScriptのスコープを理解しよう

`transition`プロパティ

JavaScriptを使ったアニメーション

プロパティ	詳細
transition-duration	変化の秒数
transition-delay	変化が開始するまでの秒数
transition-property	変化させたいプロパティ
transition-timing-function	変化の進行度

無限ループに注意

- 「画面が急に動かなくなった」「GoLiveしてもずっと読み込み中」のときは、無限ループになっている可能性がある
- 書き途中のループ文をブラウザで確認したときにも起きやすい
- 無限ループ（ブラウザが落ちる原因）の対処

JavaScriptの学び方

- 書き方は複数あり、日々アップデートされるので学び切れるものではない。無理に暗記せず、何度でも記事を見返す
- 「1つずつ理解してから次へ」では挫折する。分からないものは一旦飛ばしてよい。1〜3か月後に読み返すとあっさり理解できることも多い

- 「これは何？」と思った部分を言語化して検索する力を身につけると、自走できるようになる
- AIには、コードを書かせるのではなく、模範解答を渡して解説してもらう（初学者はコードの良し悪しを判断できないため）

> あなたはプログラミングの先生です。
> 以下のコードについて、プログラミング初学者に向けて丁寧に易しく解説してください。
>
> （ここにコード）

- 1つの記事で分からなければ「JS for文 解説」などで検索し、自分が分かりやすい記事を探してブックマークする
- 📎 変数とは何か（3つのメリット）／letとconstはどっちを使う？

【＋α】jQueryについて

- **jQuery**：JavaScriptをサイト制作用に、使いやすく簡単に書けるようにしたライブラリ。少ないコードで操作できるため、これまで多くの現場で使われてきた
- ZeroPlusで扱わない理由：将来性の低さ（2022年10月に「jQuery UI」「jQuery Mobile」が開発終了、Vue.js・React.jsなどのモダンフレームワークの台頭）
- そのため、Vue.js・React.js・jQueryの根幹となっている「JavaScript」そのものを学ぶ
- ただし現場では今も使われている場所があるので、興味があれば一度触れてみるのもよい
- 「JS アニメーション」で検索するとjQueryの記事が多く出てくる。素のJavaScript（バニラJS）で学びたいときは「素のJavaScript アニメーション」などで探す
- 📎 jQueryを勉強する準備をしよう

本日のまとめ

1. JavaScriptはCSSだけでは実現できない、ユーザーの動きに応じた「動的」な処理を可能にする
2. 変数は`let`（再代入可）・`const`（再代入不可）を使い分ける。`var`は基本的に使わない
3. `if〜else if`で条件分岐を作るときは、条件の判定順序に注意する
4. 関数は「引数を受け取り、処理をして、`return`で返す」一連の処理のまとまり
5. DOM操作は「要素を取得する → イベントを検知する → クラスを操作する」の3ステップが基本の流れ

参考記事

- JavaScriptで何が出来るか学ぼう
- JavaScriptを勉強する準備をしよう

- JavaScriptで扱えるデータの種類
- JavaScriptの変数を学ぼう
- いろいろな演算子を学ぼう
- JavaScriptの条件分岐を学ぼう
- JavaScriptの関数を学ぼう
- JavaScriptで要素を取得しよう
- JavaScriptでクラスを操作しよう
- JavaScriptのイベントでDOM操作をしよう
- JavaScriptのループ処理を学ぼう
- JavaScriptのスコープを理解しよう
- オブジェクトと配列を学ぼう
- JavaScriptで動きをつけたサイトの例
- defer属性についての参考サイト
- deferとasyncの違い（図解）
- if文の参考サイト（uxmilk）
- 引数と返り値のイメージ
- DOMとは
- transitionプロパティとは
- jQueryを勉強する準備をしよう